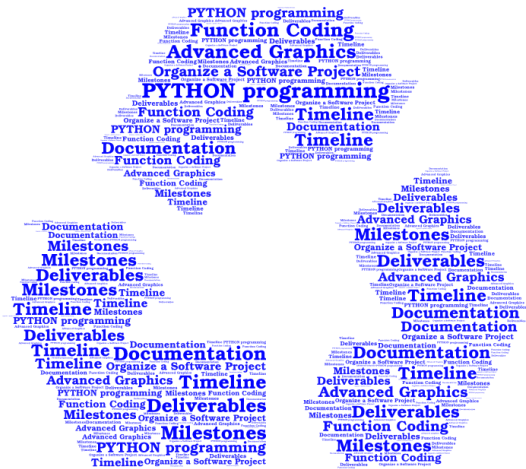




Python Programming: Work Package 1 - Scooter

Lecture: Modelling an Simulation

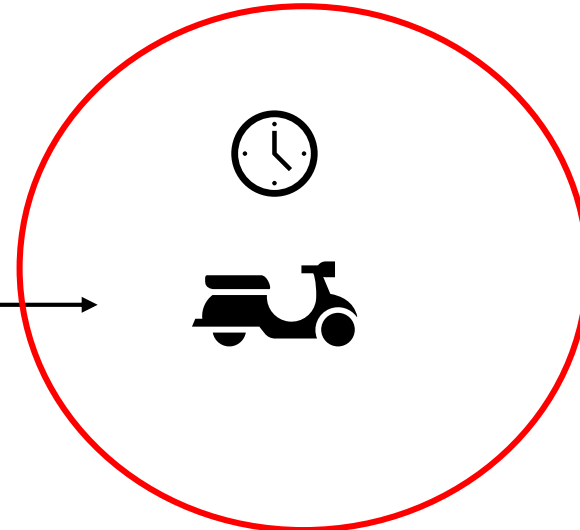
This is a compulsory elective module
of
**Technology of Circular
Economy**
(Master of Engineering)

Dr. Uwe Reimer
Senior Scientist

Overview of the project

Design a PV system which collects energy from the sun and stores it in a battery. An electric scooter shall be charged with this energy for commuting to work (daily drive cycle).

Results shall be presented for the city of Emden and second location in a different country.



Work Package 1 - Scooter

For the lecture we will use these specifications.

(Adapted from the model: Xiaomi 5 E-Scooter)

Max. speed: 20 km/h

Speed modes: 3

Max. range: 60 km

Max. load: 120 kg

Power: 350 Watt

Battery: Lithium-Ion / 48 V / 10 Ah
-> 480 Wh



- 1) Analyze driving cycle.
 - How many files are there?
 - How is the data organized in the file?
- 2) Write program to read one data file and compute:
 - Energy demand per day
 - Driving distance per day
- 3) Write program that is suitable for system simulation.
 - Needs to be called from outside the program (function or module).
 - Decide, at which point in time data is read from file (each time the function is called?).
- 4) Testing the program
- 5) Documentation

The driving cycle

The scooter collects this data every minute (over 24 h): power (in W) and speed (in km/h)
There are 7 data files (for the 7 days of the week).

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
day	1	2	3	4	5	6	7
	drive to work	drive to work	drive to work	drive to work	drive to work	drive to town	drive to town
	drive home	drive shopping	drive home	drive home	drive shopping		
		drive home			drive home	drive home	drive home

Your task: go to the Moodle system – in the lecture -> Project Scooter – D1 -> download the drive cycle (zip file). Unpack the zip file to your local directory.

The scooter collects this data every minute (over 24 h): power (in W) and speed (in km/h)

There are 7 data files (for the 7 days of the week): Each of them look like:

time / min; power / W; speed / km/h

0;0;0

1;0;0

2;0;0

...

450;80;10.0

451;80;10.0

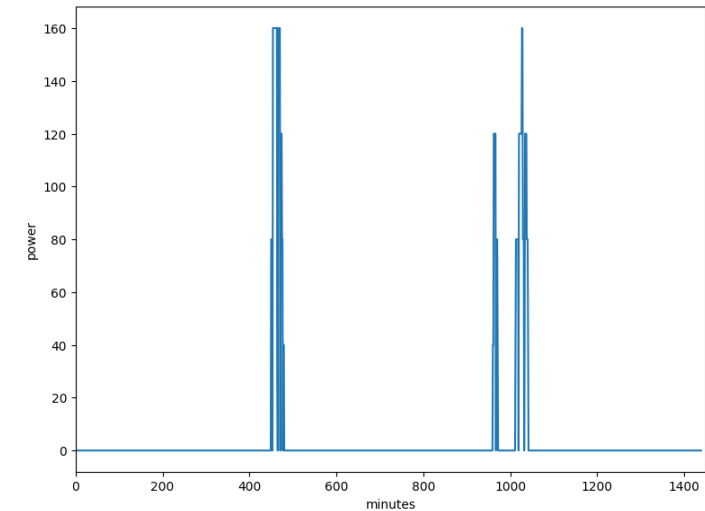
452;80;10.0

453;0;0.0

454;160;20.0

455;160;20.0

...



Work package 1 / WP 1: Scooter

Input:

- Day : **M**
- Driving Cycle : **M**
- Energy $f(\text{time})$: **S**
- Speed recorded : **C**

Output:

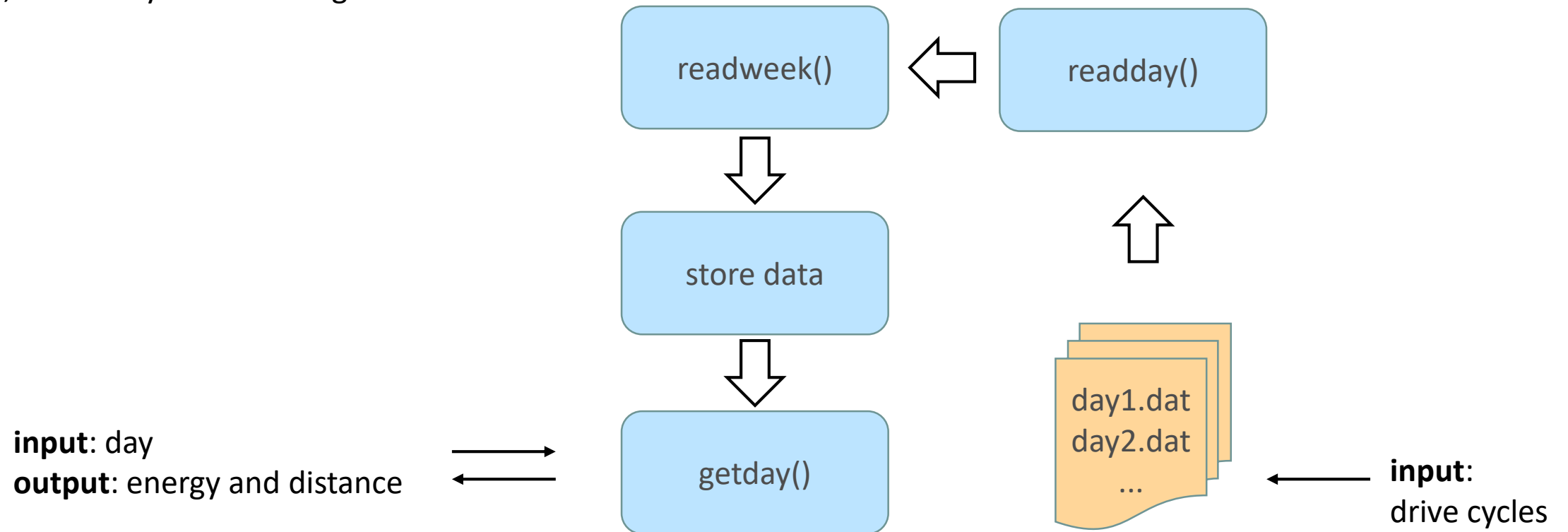
- Energy per day : **M**
- Distance driving: **C / W**

Deliverable D1: Definition of scooter energy demand per day
(scooter model ready)



Outlook: final structure

We have a set of drive cycles (7 files).
It would be good to read the files only
once, since they do not change.



The development of the model will be done in seven steps.

- 1) Reading a data file
 - 2) Converting text to numbers
 - 3) Collect numbers in lists
 - 4) Calculate sums per day
 - 5) Restructure code with function to be called from outside
 - 6) Add a function to read all files for the week and store it in a data structure
 - 7) Add a function to retrieve values for a specific day.
- (This is what we finally need.)

Step 1: reading a file

Remember from a past lecture how to open a data file.

```
8  infile = 'day1.dat' # file name
9
10 myin = open(infile, 'r') # open a text file in
11 mytext = myin.readlines() # here, we read the
12 myin.close()
13
14 print(mytext[0]) # check the list / print the
15 del mytext[0] # remove header line from list
```

The Split Function

The `split()` function lets you divide a line of text. The result is a list containing substrings. By default `split()` uses whitespace.

Example:

```
b = 'Tea bags may contain broken leaves.'  
c = b.split()
```

```
print(c)  
['Tea', 'bags', 'may', 'contain', 'broken', 'leaves.']
```

The delimiter (here whitespace) is removed in this process.
How is this useful?

The Split Function (2)

Imagine, you import a list of names, separated by comma.

Example 2:

```
b = 'James, John, Jose, Joseph'
```

Now, we split the string at the position of the comma.

```
c = b.split(',')
```

```
print(c)
```

```
['James', ' John', ' Jose', 'Joseph']
```

For our drive cycle we need to split at the semicolon.

```
# time / min; power / W; speed / km/h  
0;0;0  
1;0;0  
2;0;0  
...  
450;80;10.0  
451;80;10.0  
452;80;10.0  
453;0;0.0  
454;160;20.0  
455;160;20.0  
...
```

Step 2: Converting text to numbers

```
17     ### Step 2
18     # example
19     a = mytext[470] . . . . . # we choose entry number
20     b = a.split(';') . . . . # splitting the line in
21
22     print(b) . . . . . # see how it looks like
23     c = int(b[0]) . . . # convert text to a number
24     print(c) . . . . . # this is our first number
25
```

Step 3: Collect numbers in lists

In step 2 we learned, how to convert one number. Here, we need to convert all numbers and collect them in lists.

definition of lists

```
21  ### Step 3
22  rawmin = [] ..... # minutes - integer
23  rawpow = [] ..... # power in W - integer
24  rawspeed = [] ..... # speed in km/h - float
25  # Each entry in these lists will be for one minute.
26
27  for a in mytext:
28      .... b = a.split(';') ..... # split line of text
29      .... rawmin.append(int(b[0])) ..... # convert first text
30      .... rawpow.append(int(b[1])) ..... # convert second text
31      .... rawspeed.append(float(b[2])) ..... # convert third text
```

Step 4: Calculate sums per day

```
34     # sum over all power
35     p = 0.0
36     for a in rawpow:
37         p = p + a / 60.0 # power in Wh / 60 min = 1 hour
38
39     # calculate distance in km
40     dist = 0
41     for a in rawspeed:
42         dist = dist + a / 60.0 # 60 min = 1 hour
43
44     # output
45     s = 'Total power consumption of scooter for one day in Wh.' + '\n' + str(p)
46     print(s)
47     s = 'Total range of scooter for one day in km.' + '\n' + str(dist)
48     print(s)
```


- 1) Analyze driving cycle. 
 - How many files are there?
 - How is the data organized in the file?
- 2) Write program to read one data file and compute things. 
 - Energy demand per day
 - Driving distance per day
- 3) Write program that is suitable for system simulation.
 - Needs to be called from outside the program (function or module).
 - Decide, at which point in time data is read from file (each time the function is called?).
- 4) Testing the program
- 5) Documentation

Step 5: Restructure code

In the end we need a function that can be called from outside.
What we did until now is to calculate the sums for one day.
We take this code and turn it into a function.

```
12  infile = 'day1.dat' # file name
13
14  def readday(infile):
15      ... # read data for one day
16
17      ... myin = open(infile, 'r') # open a text file in
18      ... mytext = myin.readlines() ... # here, we read th
19      ... myin.close()
20
21      ... del mytext[0] ... # remove header line from list
22
23      ... ### Step 2
24      ... ### Step 3
```

Step 5: Restructure code

... at the end of the function we define a return value.

```
36 # sum over all power
37 p = 0.0
38 for a in rawpow:
39     p = p + a / 60.0 # power
40
41 # calculate distance in km
42 dist = 0
43 for a in rawspeed:
44     dist = dist + a / 60.0 # speed
45
46 # generate output
47 l = []
48 l.append(p)
49 l.append(dist)
50
51 return l
52 # here, l is a list and it contains
```

Step 5: Restructure code

In the end we need a function that can be called from outside.
What we did until now is to calculate the sums for one day.
We take this code and turn it into a function.

At the end we test our function.

```
54 # . check . output
55 s . = . readday (infile) . . . . # . call . function
56 print (s)
57
```

output: sum of power in Watt and distance in km

[109.33333333333339, 13.666666666666673]

Step 6: Add function to read all files

Actually, we need to read all seven files (for each day of the week).
The results should be stored in a list.

First, we create a data structure.

```
l3 myfiles = ['day1.dat', 'day2.dat', 'day3.dat', '  
l4 myresults = [] # to store results - this will be
```

Step 6: Add function to read all files

Actually, we need to read all seven files (for each day of the week).
The results should be stored in a list.

First, we create a data structure. Then we create a new function.

```
56 def readweek(myfiles):  
57     # read data for all days of the  
58     global myresults          # use th  
59  
60     day = 0  
61     while day < 7:  
62         a = readday( myfiles[day] )  
63         myresults.append(a)  
64         day = day + 1
```

loop over days

call function for one day

Step 7: Add a function to retrieve values for a specific day

Actually, this function is very simple. All it does is to look up the data for the requested day and return the two values for sum of power and sum of distance.

```
57 def readweek(myfiles):  
66 .....  
67 def getday(day):  
68     # return the sum for power and distance for a specific  
69     # day needs to be in the range of 1...7  
70     global myresults ..... # use the variable myresults.  
71     .....  
72     day = day - 1 ..... # internal count for a list starts with  
73     return myresults[day]
```

Congratulations, you have just finished the first component model for the scooter.
What is the model about?

data structure for results

data structure for input

```
14 myfiles = ['day1.dat', 'day2.dat', 'day3.dat',
15 myresults = [] # to store results - this will be
16
17 + def readday(infile):
55     ....
56 + def readweek(myfiles):
65     ....
66 + def getday(day):
```

reads data from one file
and calculates sums

reads data from all files and puts results in list

Checks in list for the sums of one specific day

Approach for this task

- 1) Analyze driving cycle. 
 - How many files are there?
 - How is the data organized in the file?
- 2) Write program to read one data file and compute things. 
 - Energy demand per day
 - Driving distance per day
- 3) Write program that is suitable for system simulation. 
 - Needs to be called from outside the program (function or module).
 - Decide, at which point in time data is read from file (each time the function is called?).
- 4) Testing the program
- 5) Documentation

Testing the scooter model

For testing we create a new file 'test.py' and save it in the same directory.

This is how the model is called later in the system simulation.

Later, it would be good to rename model from 'step07.py' into 'scooter.py' in order to gain some clarity.

```
1  import step07
2
3  # initialization
4  m = step07.myfiles
5  step07.readweek(m)
6
7
8  # check results
9  print('day .. power .. distance')
10 for a in range(7):
11     b = step07.getday(a)
12     print(a, b[0], b[1])
13
```

Testing the scooter model

For testing we create a new file 'test.py' and save it in the same directory.

This is how the model is called later in the system simulation.

Later, it would be good to rename model from 'step07.py' into 'scooter.py' in order to gain some clarity.

The output should read like this:

```
day power distance
0 109.33333333333339 13.666666666666673
1 109.33333333333339 13.666666666666673
2 115.99999999999996 14.499999999999995
3 109.33333333333339 13.666666666666673
4 109.33333333333339 13.666666666666673
5 115.99999999999996 14.499999999999995
6 109.33333333333339 13.666666666666673
```

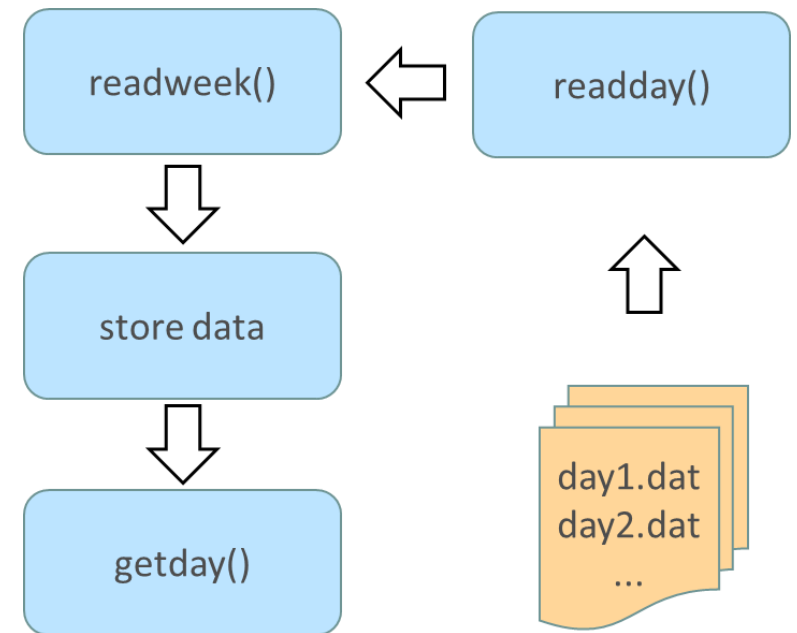
Approach for this task

- 1) Analyze driving cycle. 
 - How many files are there?
 - How is the data organized in the file?
- 2) Write program to read one data file and compute things. 
 - Energy demand per day
 - Driving distance per day
- 3) Write program that is suitable for system simulation. 
 - Needs to be called from outside the program (function or module).
 - Decide, at which point in time data is read from file (each time the function is called?).
- 4) Testing the program 
- 5) Documentation

Your job: write the software documentation for the scooter model in your final report.

Imagine, you give the program to someone else, who should use it.
Which information is needed?

- 1) Purpose of this model
- 2) What is needed for input?
 - data files: internal structure/ where should they be located/
how should they be named
- 3) What is the output? (numbers and units)
- 4) Flow chart/ workflow of the program
- 5) Test case/ example usage



Work package 1: Scooter model

1) Analyze driving cycle.



2) Write program to read one data file and compute things.



3) Write program that is suitable for system simulation.



4) Testing the program



5) Documentation



We have achieved deliverable D1!